Observability & Monitoring / SRE

L2 Interview Question Bank — Production-Focused Spoken Answers + Tricky Scenarios

Target JD: Prometheus • Grafana • ELK • OpenTelemetry • Alerting • Incident Response • Linux • Kafka/Pub-Sub

How to use: For live interviews, give the spoken answer first. Expand only when the interviewer asks for technical depth. Keep your troubleshooting structure: Impact → Triage → Metrics/Logs/Traces → Isolate → Mitigate → Recover → RCA → Prevent.

1. SRE & Observability Fundamentals

What is SRE and what are the responsibilities of an SRE?

Spoken answer: I see SRE as applying software engineering practices to production operations with reliability as a measurable outcome. I focus on SLIs and SLOs, monitoring and observability, incident response, automation, capacity and performance, safe deployments, and continuous improvement. During an incident I prioritise customer impact and service recovery, then perform RCA and preventive actions. The objective is to keep services reliable while still enabling engineering teams to deliver changes safely.

What is the difference between monitoring and observability?

Spoken answer: Monitoring tells me whether predefined conditions are healthy or unhealthy. Observability gives me enough telemetry to understand why the system is behaving that way. In practice, I use metrics to detect a problem, logs for detailed events and traces to follow a request across services, then correlate all three to isolate the root cause.

Explain the full observability flow.

Spoken answer: The application generates metrics, logs and traces. Metrics are collected and stored in Prometheus, dashboards and investigation views are provided through Grafana, logs are centralised in a platform such as ELK, and OpenTelemetry provides instrumentation and trace context. Alert rules evaluate important conditions and route actionable incidents to an on-call system such as PagerDuty or Opsgenie. Engineers then triage, mitigate, recover, validate against SLOs and perform RCA.

What are the four Golden Signals?

Spoken answer: They are latency, traffic, errors and saturation. Latency shows how long requests take, traffic shows demand, errors show failed requests, and saturation shows how close a resource or service is to capacity. I use them as a starting point and then map them to customer-facing SLIs and SLOs.

Explain RED and USE.

Spoken answer: RED is mainly for request-driven services: Rate, Errors and Duration. USE is more infrastructure-oriented: Utilization, Saturation and Errors. For an API I would start with RED, while for infrastructure resources such as CPU, memory, disk and network I use USE. Together they give me a practical view of application and platform health.

2. SLI, SLO, SLA & Reliability

Explain SLI, SLO and SLA.

Spoken answer: An SLI is the measurement, such as successful request rate or latency. An SLO is the target for that measurement, such as 99.9 percent availability. An SLA is the external contractual commitment to a customer. I use SLOs to drive monitoring and alerting, while the error budget helps balance reliability work against feature delivery.

What is an error budget?

Spoken answer: An error budget is the amount of unreliability permitted by the SLO. With a 99.9 percent availability SLO, the remaining 0.1 percent represents the budget. I use its consumption as a reliability signal. If the budget is being consumed too quickly, I would prioritise reliability work and investigate the causes rather than continuing to introduce unnecessary risk.

Would you alert directly on SLO violation?

Spoken answer: Not always. I distinguish between fast-burn conditions that threaten the SLO quickly and slower conditions that can be handled through trend monitoring. For paging, I want a condition that requires immediate human action. For example, a short but severe error spike can page immediately, while a gradual budget burn may create a ticket or lower-severity notification.

What is the difference between availability and reliability?

Spoken answer: Availability is whether the service is usable at a point in time or over a defined period. Reliability is broader and includes the probability of performing correctly over time. A service can be highly available but still unreliable if it frequently returns incorrect results, has intermittent failures, or violates latency expectations.

3. Prometheus & PromQL

Explain Prometheus architecture.

Spoken answer: Prometheus is primarily pull based. Targets expose metrics, Prometheus discovers those targets and scrapes them at intervals, then stores the resulting time series with labels. PromQL is used for querying and alert rules. Alertmanager handles alert grouping, deduplication, routing and notification. In Kubernetes, service discovery and exporters help monitor nodes, pods, containers and applications.

What are the four Prometheus metric types?

Spoken answer: The main metric types are Counter, Gauge, Histogram and Summary. Counters represent cumulative events such as requests or errors. Gauges represent current values such as CPU or queue depth. Histograms capture distributions such as request latency and can be aggregated across instances. Summaries calculate quantiles on the client side.

Give a PromQL query for error rate.

Spoken answer: I would calculate failed request rate divided by total request rate. For example: sum(rate(http_requests_total{status=~"5.."}[5m])) divided by sum(rate(http_requests_total[5m])). I would then adapt the labels and window to the actual application.

How do you monitor P95 latency with Prometheus?

Spoken answer: If the application exposes a histogram, I use histogram_quantile over the rate of the bucket series. For example: histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m]))). I validate that the histogram buckets are appropriate for the service's latency range.

Prometheus has stopped scraping one service. How do you troubleshoot?

Spoken answer: I first check the target status and scrape errors. Then I verify that the application's metrics endpoint is reachable and returning valid metrics. In Kubernetes I check service discovery, ServiceMonitor or scrape configuration, labels, endpoints and network policies. I also check Prometheus logs and recent configuration changes. Finally I validate that samples are arriving again.

What is high cardinality and why is it dangerous?

Spoken answer: High cardinality means a metric has too many unique label combinations, such as putting request IDs or user IDs into labels. That creates a large number of time series and can consume significant memory and storage. I keep labels bounded and avoid unbounded identifiers. For request-level investigation, I use logs or traces instead of metric labels.

4. Grafana

How would you design a production Grafana dashboard?

Spoken answer: I start with the critical user journey and put the most important signals at the top: availability, traffic, error rate and P95 or P99 latency. Then I show saturation, dependency health, infrastructure, deployment markers and regional comparisons. I add environment, service and region filters. The dashboard should help an on-call engineer quickly answer what is broken, who is affected and where to investigate.

Grafana says No Data. What do you check?

Spoken answer: I determine whether the data exists in the underlying data source first. If Prometheus has no data, I investigate scraping or instrumentation. If Prometheus has the data, I check the Grafana data source, time range, query, variables and dashboard configuration. I avoid assuming that No Data means the application is healthy.

What is the difference between a dashboard and an alert?

Spoken answer: A dashboard is primarily for investigation and visualisation. An alert is an automated signal that requires a response. A dashboard can contain many useful metrics, but only a subset should generate pages. I design alerts around actionable symptoms and customer impact rather than turning every dashboard threshold into a page.

5. Alert Engineering & PagerDuty/Opsgenie

How do you design an actionable alert?

Spoken answer: I start with a condition that requires human action. I define the signal, threshold or SLO condition, evaluation window, severity, ownership, routing and runbook. I use grouping and deduplication so one incident doesn't generate dozens of pages. I then review alert history and tune it based on false positives, missed incidents and operator feedback.

You receive 40 alerts per day and 75 percent are non-actionable. What do you do?

Spoken answer: I analyse several weeks of alert history and classify alerts as actionable, informational or noise. I look for poor thresholds, short evaluation windows, duplicate symptoms and alerts that self-resolve without intervention. I tune thresholds and durations, introduce grouping and deduplication, change severity or routing where appropriate, and remove alerts that provide no operational value. I then validate that real incidents are still detected.

How do you decide whether CPU should alert at 80, 85 or 90 percent?

Spoken answer: I don't choose the number arbitrarily. I use historical baselines, workload behaviour, resource capacity and correlation with customer-facing SLIs. If latency starts degrading around 80 percent CPU, that can justify an actionable threshold. If the service remains healthy until 90 percent, 80 percent may be informational. The threshold should reflect observed system behaviour and the consequence of saturation.

What is the difference between a symptom alert and a cause alert?

Spoken answer: A symptom alert detects customer or service impact, such as high error rate or SLO burn. A cause alert detects a contributing condition, such as CPU saturation. I prioritise actionable symptom alerts for paging because they represent impact. Cause signals can support diagnosis or lower-severity notification. This reduces alert storms where many underlying metrics page for the same incident.

How do grouping and deduplication help?

Spoken answer: Grouping combines related alerts into one incident, while deduplication prevents repeated copies of the same alert from generating multiple notifications. For example, if one database outage causes failures in 20 services, I want an incident with useful context rather than 20 separate pages. I still preserve enough detail for responders to identify affected services.

What should PagerDuty or Opsgenie escalation contain?

Spoken answer: I would define service ownership, severity, primary and secondary responders, escalation timing, acknowledgement requirements, maintenance or suppression rules, and links to dashboards and runbooks. The

escalation policy should make it clear who is expected to respond and what happens when an alert isn't acknowledged.

6. Application Performance & Troubleshooting

An application is slow but dashboards are green. How do you investigate?

Spoken answer: I start with the user transaction rather than assuming the infrastructure is healthy. I check request rate, errors and P95/P99 latency, then use distributed traces to identify slow spans. I correlate the trace with application logs and check database, cache, gateway, network and third-party dependencies. I also compare the timing with recent deployments or configuration changes. Once isolated, I mitigate and validate the same user-facing SLI.

API latency suddenly increases. Walk me through it.

Spoken answer: First I establish impact and scope: service, region, customer segment and time window. Then I check traffic, error rate, P95/P99 latency and saturation. I correlate the timing with deployments and configuration changes. Next I inspect traces for slow downstream calls and logs for errors or timeouts. I check databases, caches, gateways, network and dependencies. I mitigate through rollback, scaling or dependency recovery as appropriate, then validate recovery and complete RCA.

How do you distinguish CPU saturation from a database problem?

Spoken answer: I correlate the latency increase with both CPU and database metrics rather than using one signal. If CPU saturation rises at the same time as request latency and application processing time, CPU may be the bottleneck. If application spans show database time dominating the request, I investigate database latency, connection pools, locks and query performance. I also compare healthy and affected instances or regions.

Everything is green but customers in one region report failures. What do you do?

Spoken answer: I compare the affected region against a healthy region. I check regional traffic, latency, error rate, DNS, load balancing, networking, dependencies and deployment versions. I filter logs and traces by region to identify the failing layer. If the architecture supports it, shifting traffic to a healthy region may be a mitigation. Afterwards I investigate why regional detection or alerting did not trigger earlier.

A third-party API becomes slow. How do you handle it?

Spoken answer: I first establish how much customer impact it causes and whether the latency is isolated to that dependency. Traces can show the dependency span duration, while logs show timeouts and errors. I check whether there is a timeout, retry or circuit-breaker policy. Depending on the application, mitigation could involve failover, fallback behaviour, reduced retries or routing to an alternate provider. I also make sure retries aren't amplifying the incident.

7. OpenTelemetry & Distributed Tracing

What is OpenTelemetry?

Spoken answer: OpenTelemetry is an open-source framework for generating and collecting telemetry, including traces, metrics and logs. I particularly use it for distributed tracing, where trace context follows a request across services. That allows me to identify which service, database or downstream dependency is responsible for latency or failure.

How do you troubleshoot a slow request using tracing?

Spoken answer: I locate a slow trace and inspect its spans. I compare the total duration with individual downstream spans to find the largest contributor. Then I correlate the trace ID with application logs and Prometheus metrics. If a database span consumes most of the time, I investigate queries and connection pools; if another service is slow, I move into that service's telemetry. After mitigation I validate latency against the SLO.

What is trace context propagation?

Spoken answer: It is the mechanism that carries tracing information, such as trace and span context, between service boundaries. Without propagation, I may see separate traces rather than one end-to-end transaction. Proper propagation allows me to follow a request across APIs, queues and downstream services where supported.

Metrics show high latency but traces look normal. What could explain that?

Spoken answer: I would check whether the instrumentation covers the complete transaction and whether sampling is hiding affected requests. I would also compare the metric definition with the trace duration, check ingress or gateway latency outside the application spans, and look for queueing, client-side latency or regional network effects. The key is to validate that the two signals measure the same transaction boundary.

8. ELK / Logging

Explain Elasticsearch, Logstash and Kibana.

Spoken answer: Logstash is commonly used to ingest and transform logs, Elasticsearch stores and indexes the data, and Kibana provides search and visualisation. In production troubleshooting I would filter by service, environment, timestamp, severity and correlation or trace ID, then correlate the results with metrics and traces.

You have not worked extensively with ELK. How would you answer honestly?

Spoken answer: My strongest hands-on logging experience has been with Splunk and Azure Log Analytics rather than ELK. I understand the ELK architecture and the operational workflow, and the underlying troubleshooting concepts are transferable. I would be comfortable learning the specific ingestion, indexing and Kibana query patterns used by the team.

Elasticsearch disk usage reaches 90 percent. What do you investigate?

Spoken answer: I check index growth, retention policies, shard allocation, replica usage and whether a high-volume application is generating excessive logs. I also check cluster health and disk watermarks. I would avoid simply deleting data without understanding retention requirements. Mitigation could include removing eligible old indices, adjusting retention, controlling log volume or expanding storage according to policy.

Logs are missing from Kibana. What is your troubleshooting flow?

Spoken answer: I trace the ingestion path: application or agent, Logstash or collector, Elasticsearch ingestion and finally Kibana query/index pattern. I check timestamps, parsing errors, pipeline failures, network connectivity and index availability. I also verify whether the query is using the correct data view and time range. This tells me whether the problem is generation, transport, indexing or visualisation.

9. Linux & Platform Administration

How do you investigate high CPU on Linux?

Spoken answer: I confirm the condition with top or ps, identify the process and inspect its CPU behaviour. I then check application logs, load average, memory, I/O and recent changes. If necessary I apply the approved mitigation such as restarting or scaling the service, then verify recovery. Afterwards I investigate why CPU increased so the problem doesn't recur.

How do you find the top five CPU-consuming processes?

Spoken answer: I can use `ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -6`. I can also use top and sort by CPU interactively.

How do you investigate high memory?

Spoken answer: I identify the process and its RSS, check whether memory is steadily increasing, inspect application logs and correlate with workload. I check system memory and swap as well as container limits if applicable. In Kubernetes I specifically check for OOMKilled events, requests and limits, restart history and memory metrics.

How do you investigate a full filesystem?

Spoken answer: I use `df -h` to identify the affected filesystem and `du` to locate large directories. I then check logs, temporary files, old artifacts and deleted-but-open files. I clean or rotate data according to operational policy rather than deleting files blindly. I also identify why the growth occurred and add capacity or retention controls if needed.

What is load average?

Spoken answer: Load average represents the number of tasks that are runnable or waiting for uninterruptible I/O over one, five and fifteen minutes. I interpret it together with CPU count and CPU utilisation. A high load average with low CPU can indicate I/O or other wait conditions rather than simply high CPU usage.

10. Kafka / Pub-Sub / Distributed Systems

Explain Kafka at an awareness level.

Spoken answer: Producers publish messages to topics, topics are divided into partitions, brokers store those partitions, and consumers read messages. Consumer groups allow multiple consumers to process partitions in parallel, and consumers track offsets. From an SRE perspective I monitor broker health, partition availability, throughput, consumer lag, disk usage and errors.

Consumer lag suddenly increases. What do you check?

Spoken answer: I determine whether lag is isolated to one consumer group, topic or partition. Then I check consumer health, processing rate, restarts and errors. I check partition distribution and broker health, including CPU, disk and network. I also check whether producer traffic increased or a downstream dependency slowed processing. The goal is to distinguish increased demand from insufficient consumer capacity, partition imbalance or downstream latency.

What can cause duplicate message processing?

Spoken answer: Possible causes include consumer retries, reprocessing after failures, offset commit timing and at-least-once delivery semantics. I would check the consumer's offset and acknowledgement or commit behaviour. Where business correctness requires it, consumers should be designed to be idempotent so processing the same message twice doesn't create an incorrect outcome.

What happens when a Kafka consumer goes down?

Spoken answer: Kafka can rebalance the consumer group and assign partitions to remaining consumers, depending on the group configuration. If capacity is insufficient, lag can grow. I check consumer-group membership, partition assignment, lag, errors and broker health, then restore or scale consumers as appropriate.

What is backpressure in a distributed system?

Spoken answer: Backpressure occurs when a downstream component cannot process data as quickly as the upstream component produces it. Symptoms can include growing queues, Kafka lag, increased latency and resource saturation. I look for the slow component, control the input rate if possible, scale consumers or workers, and make sure retries don't amplify the load.

11. Incident Response & RCA

Walk me through a P1 incident.

Spoken answer: I acknowledge the incident and establish impact and scope. I bring in the required teams and establish clear ownership and communication. I use metrics, logs and traces to identify the failing layer while keeping mitigation as the immediate priority. Depending on the situation I may rollback, scale, fail over or restore a dependency. After recovery I validate SLIs, document the timeline and perform RCA with corrective and preventive actions.

What is the difference between mitigation and resolution?

Spoken answer: Mitigation restores or stabilises service without necessarily fixing the underlying cause. For example, scaling or rollback may restore service. Resolution addresses the root cause. I treat mitigation as the immediate objective and follow it with a permanent corrective action.

What should a runbook contain?

Spoken answer: It should contain the alert, symptoms, impact, prerequisites, dashboards, logs, diagnostic commands, decision points, mitigation and rollback steps, escalation contacts and validation checks. I also include dependency information and links to relevant systems. The goal is to make the procedure executable by another engineer during a high-pressure incident.

What makes a good RCA?

Spoken answer: A good RCA explains the customer impact, timeline, technical root cause, contributing factors, detection and response gaps, mitigation and permanent corrective actions. I focus on system and process improvements rather than individual blame. Actions should have owners and measurable completion criteria.

How do you reduce MTTR?

Spoken answer: I improve detection quality, telemetry, alert routing, runbooks, ownership and automation. During incidents I use structured triage and clear roles to reduce duplicated effort. After incidents I remove recurring failure modes and automate repetitive recovery tasks where safe. The objective is faster detection, diagnosis, mitigation and recovery—not simply faster alerting.

12. Tricky L2 Scenario Questions

A deployment is successful according to CI/CD, but users immediately report failures. What do you do?

Spoken answer: I treat deployment success and service health as different signals. I check application error rate, latency, logs and traces and compare the new version with the previous version. I inspect health checks and dependency compatibility. If the deployment correlates strongly with impact and rollback is safe, I roll back to restore service. Then I investigate the deployment failure mode and add a test, gate or observability improvement.

Prometheus shows 99.9 percent availability, but customers say the service is unavailable. How do you investigate?

Spoken answer: I first validate what the availability metric actually measures. It may be measuring only HTTP success from one probe location while customers are failing elsewhere. I compare multiple regions, ingress or gateway metrics, real-user signals and synthetic checks. I verify the metric query, labels and aggregation. This is a case where a technically correct metric can still fail to represent the customer journey.

CPU is 95 percent, but latency and errors are normal. Do you page?

Spoken answer: Not automatically. I would determine whether 95 percent CPU is expected for that workload and whether there is any customer or SLO impact. If the service has enough headroom and historical behaviour shows no degradation, I may use a warning or capacity alert rather than a page. I would still investigate whether the system is approaching a saturation point.

Latency is high but CPU is only 30 percent. What could be happening?

Spoken answer: I would investigate dependencies and non-CPU bottlenecks: database latency, network latency, connection pools, locks, external APIs, disk I/O, thread pools, queueing and rate limits. I would use traces to identify where request time is actually spent instead of assuming low CPU means the application is healthy.

Error rate is normal but users report intermittent failures. What could be wrong?

Spoken answer: I would check whether the error metric is too broad or sampled incorrectly. I would segment by region, endpoint, status code, client, dependency and instance. I would inspect traces and logs for timeouts or retries and compare healthy versus affected requests. Intermittent issues can also come from one unhealthy instance, load-balancer routing, DNS, connection pools or a specific dependency.

Your alert fires every five minutes and auto-resolves. How do you decide whether to change it?

Spoken answer: I review whether the alert requires human action and whether it correlates with impact. If it is a transient condition with no user impact, I may increase the evaluation window, use a rate or burn-rate condition, lower the severity or remove paging. If the transient event is an early indicator of a serious failure, I retain it but route it appropriately. The decision is based on operational value, not simply alert frequency.

A single bad instance causes errors while the service overall looks healthy. How do you detect it?

Spoken answer: I use instance-level or pod-level labels and compare error rate and latency across instances. Load balancer metrics and traces can identify whether traffic is disproportionately reaching one instance. I would check that instance's logs, resources, deployment version and health. This is why aggregate service metrics should be complemented by appropriate dimensional analysis.

You have a dashboard with 200 panels. Is it a good dashboard?

Spoken answer: Not necessarily. A production dashboard should prioritise the signals needed to make decisions. I would separate executive or service-health views from detailed troubleshooting dashboards. The primary dashboard should surface the critical user journey, SLOs, RED or Golden Signals and key dependencies, with drill-down dashboards for deeper investigation.

A Kafka topic has healthy brokers but consumer lag keeps increasing. What is your hypothesis?

Spoken answer: I would suspect consumer-side throughput or a downstream bottleneck before assuming broker failure. I check consumer processing rate, consumer restarts, partition assignment, CPU and memory, downstream database or API latency, and producer rate. If producer throughput increased beyond consumer capacity, scaling consumers may help, provided partitions allow additional parallelism.

OpenTelemetry traces increase latency in production. What do you investigate?

Spoken answer: I would check instrumentation overhead, sampling configuration, exporter behaviour and network or collector performance. Tracing should be designed so telemetry overhead remains bounded. I would compare application latency with tracing enabled and disabled where safe, review sampling rates, and make sure exporters are not blocking critical application paths.

The monitoring system itself is down during an application incident. What do you do?

Spoken answer: I treat observability as a production dependency. I use alternate signals such as application logs, cloud-native monitoring, host-level checks or synthetic tests to establish impact. In parallel I restore the monitoring platform. Afterward I address monitoring high availability, independent monitoring paths and monitoring-the-monitoring controls.

A customer says only one API endpoint is slow while infrastructure and overall service latency are normal. What do you check?

Spoken answer: I segment metrics by endpoint and use traces for that transaction. I check endpoint-specific database queries, downstream calls, payload size, cache behaviour and application code paths. Aggregate service metrics can hide an endpoint-specific problem, so the telemetry needs the right dimensions.

You see thousands of alerts during one outage. What probably happened?

Spoken answer: Most likely several alerts represent the same underlying incident. I would look for missing grouping, deduplication or dependency-aware routing. I would identify the primary customer-impacting symptom and treat the other alerts as supporting signals. After recovery, I would redesign the alert relationships so one root event does not create an alert storm.

A service has zero traffic overnight. Its request-rate alert fires because the value is zero. Is that a valid alert?

Spoken answer: Not necessarily. Zero traffic may be completely expected. I would first understand the service's normal operating schedule. If traffic is expected, the alert is useful only when zero traffic itself represents an abnormal condition. Alert logic should account for expected behaviour rather than treating every zero value as an incident.

A metric suddenly disappears. Does that mean the application is healthy?

Spoken answer: No. Missing telemetry is itself a potential observability failure. I check the scrape target, exporter, instrumentation, service discovery and network path. I also distinguish between zero and missing data. A missing metric can hide a real production incident, so important telemetry pipelines need their own health monitoring.

13. Quick Commands & Cheat Sheet

Top CPU processes

Spoken answer: `ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -6`

Top memory processes

Spoken answer: `ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -6`

Disk usage

Spoken answer: `df -h`

Directory sizes

Spoken answer: `du -sh /var/log/*`

Large files

Spoken answer: `find /var -type f -size +1G -ls`

Process lookup

Spoken answer: `ps -ef | grep <process>` or `pgrep -af <process>`

Terminate process

Spoken answer: `kill <PID>`; use `kill -9 <PID>` only when graceful termination does not work and the impact is understood.

Recent logs

Spoken answer: `journalctl -u <service> --since "30 min ago"`

Open ports

Spoken answer: `ss -lntp`

Cron fields

Spoken answer: Minute, Hour, Day of Month, Month, Day of Week. `*/5 * * * *` means every five minutes.

Kubernetes quick triage

Spoken answer: `kubectl get pods -A`; `kubectl describe pod <pod>`; `kubectl logs <pod> --previous`; `kubectl get events --sort-by=.lastTimestamp`

Git workflow

Spoken answer: `git status`; `git pull`; create a feature branch; edit; `git diff`; `git add`; `git commit`; `git push`; raise a PR.

AWS Logs Insights

Spoken answer: `fields @timestamp, @message | filter @message like /critical/ | sort @timestamp desc | limit 100`

Final L2 Interview Strategy

Answer structure: Impact → Triage → Metrics → Logs → Traces → Isolate → Mitigate → Recover → RCA → Prevent.

For observability: Always connect telemetry to a user journey and SLO. Avoid saying that CPU, memory or infrastructure health alone proves application health.

For alerting: Explain why the alert requires action, how you selected the threshold or burn-rate condition, how you avoid duplicate pages, and what the responder does next.

For unfamiliar tools: Be transparent. A strong answer is: “I haven't worked extensively with that specific tool, but based on my SRE/DevOps experience, the operational approach would be…” Then explain the architecture and troubleshooting method.

For production incidents: Do not jump straight to scaling. First establish impact, form a hypothesis using telemetry, isolate the failing component, choose the safest mitigation, validate recovery and then address the root cause.

Most important topics to revise: PromQL, histogram/P95/P99, SLO/error budget, alert fatigue, burn-rate alerting, OpenTelemetry trace correlation, ELK fundamentals, Linux troubleshooting, Kafka lag and incident response.

Important accuracy point: Do not claim hands-on ELK, PagerDuty/Opsgenie or Kafka experience beyond what you have actually done. Present awareness and transferable SRE experience clearly.